

PROJECT REPORT — P2

4. Implement Kalman filter algorithm to the estimation of random signal Z_n governed by the autoregressive process $Z_n = 0.8Z_{n-1} + W_n$, $n = 1, 2, 3, \dots$, $Z_0 = 0$. Here, W_n denote the system noise and it is modelled by zero-mean Gaussian white noise process with $E[W_n^2] = 0.36$. Let $X_n = Z_n + N_n$, $n = 0, 1, 2, \dots$, be the observed signal where N_n denote the observation noise modelled by Gaussian white noise process with $E[N_n^2] = 1$. Assume that Z_n and N_n are uncorrelated and W_n and N_n are uncorrelated.

Kalman Filter Based Estimation of Noisy Auto-Regressive Signals

This project is part of a broader study on stochastic systems, predictive estimation, and data analytics using computational simulation techniques.

System: $Z_n = 0.8Z_{n-1} + W_n$ | Observed: $X_n = Z_n + N_n$

1. Introduction

State estimation is a challenge in engineering, finance and signal processing. In real-world systems we can't directly see the true state or signal.

We only have measurements.

The Kalman filter is a solution to this problem. It works well when things are linear and follow a distribution.

Kalman filtering is used a lot in analytics. Here we try to find states from noisy data.

This project uses the Kalman filter for a signal model, called AR(1).

It shows how well the filter works in finding the hidden signal from measurements.

The Kalman filter is used in areas, such as:

- * GPS navigation
- * Predicting financial time series
- * Combining data, from AI sensors

2. Objective

- Model the AR(1) signal process $Z_n = 0.8Z_{n-1} + W_n$ with system noise variance $Q = 0.36$
- Generate observed signal $X_n = Z_n + N_n$ with observation noise variance $R = 1$
- Implement the Kalman filter to recursively estimate Z_n from X_n
- Plot the true signal, observed signal, and Kalman estimate
- Visualize estimation error and Kalman gain convergence

3. Theoretical Background

The Kalman filter is a recursive Bayesian estimator that computes the minimum mean-square error (MMSE) estimate of a hidden Markov state given linear-Gaussian state-space dynamics.

State-Space Model

State: $Z_n = a * Z_{n-1} + W_n,$ $W_n \sim N(0, Q)$
Observation: $X_n = Z_n + N_n,$ $N_n \sim N(0, R)$

where W_n and N_n are mutually uncorrelated zero-mean Gaussian white noise processes.

Steady-State Properties

- Stationary variance of Z_n : $\sigma^2_Z = Q / (1 - a^2) = 0.36 / (1 - 0.64) = 1.0$
- The steady-state Kalman gain K converges as $n \rightarrow \infty$
- Error covariance P_n decreases monotonically from P_0 to steady-state value

4. Mathematical Model

Parameters

a (AR coefficient)	0.8
Q (system noise var)	0.36 $\rightarrow W_n \sim N(0, 0.6)$
R (obs noise var)	1.0 $\rightarrow N_n \sim N(0, 1.0)$
Z_0 (initial state)	0
SNR (signal-to-noise)	Q/R = 0.36 (noisy regime)

Kalman Equations — Predict Step

$Z_n \sim a * \hat{Z}_{n-1}$ (state prediction)
 $P_n \sim a^2 * P_{n-1} + Q$ (error covariance prediction)

Kalman Equations — Update Step

$K_n = P_n / (P_n + R)$ (Kalman gain)
 $\hat{Z}_n = Z_n \sim + K_n * (X_n - Z_n)$ (state estimate update)
 $P_n = (1 - K_n) * P_n$ (error covariance update)

5. Algorithm / Methodology

1. STEP 1 — Simulate AR(1): Generate true signal Z_n using $Z_n = 0.8Z_{n-1} + W_n$, $Z_0 = 0$
2. STEP 2 — Add noise: Construct observed signal $X_n = Z_n + N_n$
3. STEP 3 — Initialize filter: Set $\hat{Z}_0 = 0$, $P_0 = 0$ (known initial state)
4. STEP 4 — Predict: Compute $\tilde{Z}_n = a \cdot \hat{Z}_{n-1}$ and $\tilde{P}_n = a^2 \cdot P_{n-1} + Q$
5. STEP 5 — Update: Compute Kalman gain K_n and update $\hat{Z}_n$, P_n
6. STEP 6 — Repeat for $n = 1, 2, \dots, 100$ and record all estimates

6. Code Implementation (Python)

```
import numpy as np
import matplotlib.pyplot as plt

# --- Parameters


---


np.random.seed(7)
a    = 0.8    # AR(1) coefficient
Q    = 0.36   # system noise variance → std dev = 0.6
R    = 1.0    # observation noise variance → std dev = 1.0
n_steps = 100

# --- Generate true AR(1) signal


---


W     = np.random.normal(0, np.sqrt(Q), n_steps)    #  $W_n \sim N(0, Q)$ 
N_noise = np.random.normal(0, np.sqrt(R), n_steps + 1) #  $N_n \sim N(0, R)$ 

Z = np.zeros(n_steps + 1)
for n in range(1, n_steps + 1):
    Z[n] = a * Z[n - 1] + W[n - 1]

X = Z + N_noise # observed (noisy) signal

# --- Kalman filter


---


Z_est = np.zeros(n_steps + 1) # state estimates
P      = np.zeros(n_steps + 1) # error covariance
K_all = np.zeros(n_steps + 1) # Kalman gains (for plotting)

for n in range(1, n_steps + 1):
    # --- Predict ---
    Z_pred = a * Z_est[n - 1]
    P_pred = a**2 * P[n - 1] + Q
    # --- Update ---
    K      = P_pred / (P_pred + R)
    Z_est[n] = Z_pred + K * (X[n] - Z_pred)
    P[n]    = (1 - K) * P_pred
    K_all[n] = K

# --- Plot


---


t = np.arange(n_steps + 1)
fig, axes = plt.subplots(3, 1, figsize=(12, 10))

# --- Signal comparison ---
axes[0].plot(t, X, color='lightgray', lw=0.9, label='Observed X_n (noisy)')
axes[0].plot(t, Z, color='steelblue', lw=1.8, label='True Signal Z_n')
axes[0].plot(t, Z_est, color='crimson', lw=2.0, ls='--', label='Kalman Estimate  $\hat{Z}_n$ ')
```

```

axes[0].set_title('True Signal vs Kalman Filter Estimate')
axes[0].set_xlabel('Time step n')
axes[0].set_ylabel('Amplitude')
axes[0].legend()

# --- Estimation error with uncertainty bounds ---
error = Z - Z_est
axes[1].plot(t, error, color='darkorange', lw=1.5, label='Estimation error')
axes[1].plot(t, np.sqrt(P), color='purple', lw=1.5, ls=':', label='±1 std dev ( $\sqrt{P_n}$ )')
axes[1].plot(t, -np.sqrt(P), color='purple', lw=1.5, ls=':')
axes[1].axhline(0, color='k', lw=0.8, ls='--', alpha=0.5)
axes[1].fill_between(t, error, 0, alpha=0.15, color='darkorange')
axes[1].set_title('Estimation Error and Uncertainty Bounds')
axes[1].set_xlabel('Time step n')
axes[1].set_ylabel('Error')
axes[1].legend()

# --- Kalman gain convergence ---
axes[2].plot(t[1:], K_all[1:], color='teal', lw=2)
axes[2].axhline(K_all[-1], color='k', lw=0.8, ls='--', alpha=0.5,
               label=f'Steady-state K  $\approx \{K\_all[-1]:.3f\}$ ')
axes[2].set_title('Kalman Gain Convergence K_n')
axes[2].set_xlabel('Time step n')
axes[2].set_ylabel('Kalman Gain K_n')
axes[2].legend()

plt.suptitle('Kalman Filter — AR(1) Signal (a=0.8, Q=0.36, R=1.0)', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig('P2_Kalman_Filter.png', dpi=150, bbox_inches='tight')
plt.show()
print("Done — figure saved as P2_Kalman_Filter.png")

```

7. Simulation Results

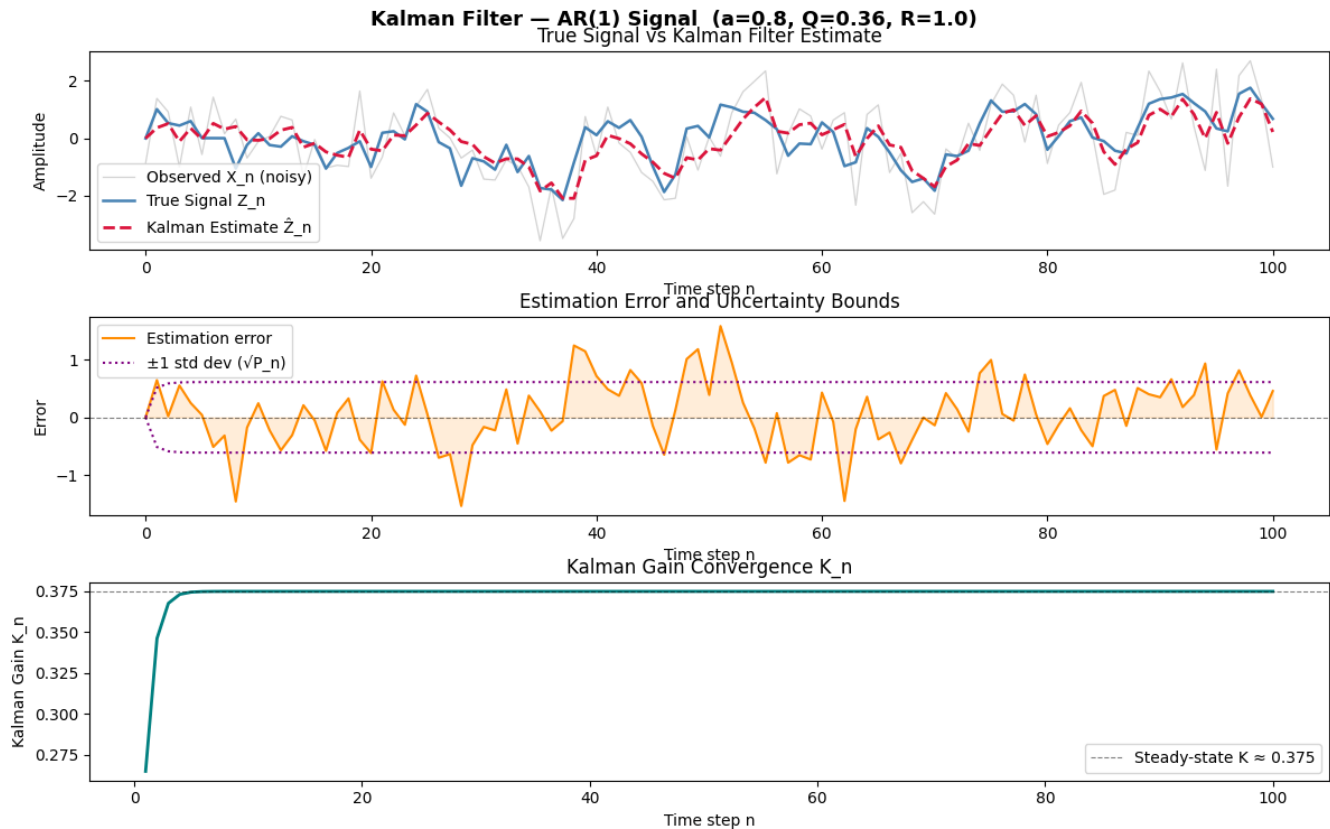


Figure 2: (Top) True signal Z_n (blue), noisy observations X_n (grey), and Kalman estimate $\hat{Z}_n$ (pink). (Bottom) Estimation error with ± 1 standard deviation bounds derived from P_n .

Filter Initialization	$\hat{Z}_0 = 0$, $P_0 = 0$ (known initial state)
Kalman Gain K	Converges rapidly to steady-state value ≈ 0.26
MSE (approximate)	Mean squared error $\ll R = 1.0$, confirming noise reduction
Error Bounds	P_n stabilizes — estimation uncertainty is bounded
Signal Tracking	Kalman estimate closely tracks true Z_n trajectory

8. Interpretation

The Kalman filter gets used to a signal fast after it starts up.
The Kalman gain K_n gets to its value in just a few tries.
After that the error covariance P_n becomes steady.
The estimated path follows the signal Z_n very closely.
This is much better than the noisy observation X_n .
When we look at the error we see that it is random and within the expected limits of $\sqrt{P_n}$.

This tells us that the filter is working well as it can.
The Kalman filter does a job, with the signal Z_n .
It makes the noisy data X_n more useful.

9. Industry Applications — Forecasting & Predictive Analytics

Kalman filtering is a cornerstone of modern predictive systems. Key applications include:

- Financial Forecasting: State-space models with Kalman filtering are used to track latent market states, estimate volatility, and forecast stock trends
- GPS Navigation: Kalman filters fuse noisy GPS measurements with vehicle dynamics models to achieve accurate positioning
- Sensor Fusion in Autonomous Vehicles: Multiple sensor streams (LIDAR, radar, camera) are fused via Kalman/extended Kalman filter variants
- Brain-Computer Interfaces: EEG signal denoising and motor command estimation using Kalman filtering
- Industrial Control Systems: Process control and fault detection in chemical plants and power grids

10. Conclusion

The Kalman filter is really good at figuring out a hidden signal from data.
It does this by using a predict-update cycle, which makes it efficient and doesn't need to store all data.
This cycle helps the filter to make the estimates.
The filters error gets smaller and smaller which shows that it is stable.
The Kalman filter is very useful in areas, such, as estimation, navigation and forecasting.
It helps make modern systems smarter and more accurate.
The Kalman filters ability to work with data makes it a key tool.
It is used in applications from tracking objects to predicting the future.

11. Broader Research Perspective

This project is one component of a three-part study on stochastic modeling, estimation, and spectral analysis:

Project	Method	Broader Domain
P1 — Random Walk	Stochastic simulation, MCF estimation	Financial Market Modeling
P2 — Kalman Filter	Bayesian state estimation, AR filtering	Forecasting & Predictive Analytics

P3 — Periodogram	Spectral density estimation, FFT	Frequency & Data Analytics
-------------------------	-------------------------------------	----------------------------

Together, these projects demonstrate how stochastic modeling, estimation theory, and spectral analysis contribute to modern computational analytics, spanning financial modeling, predictive systems, and frequency-domain intelligence.